

README

Part 19: Agent 与 Function Calling — 手写 Agent Loop、MCP 协议与 τ -bench 评测（应用线 A2）

- > [Part 17](../Part17_agentic_rl/tutorial/README.md) 是本部分的训练侧姊妹篇：
- > 17 把多轮工具调用轨迹当训练数据做 RL（训 agent），19 把同一条轨迹在推理侧
- > 跑起来（用 agent）。一个 **while** 循环 + 三工具 + 三种终止条件就是 agent 的
- > 全部骨架（[脚本 01](../scripts/01_agent_loop.py)，GPU 实测 ~20 秒）；在此之上
- > 补齐工程生态：工具层协议 MCP 手写复刻（[脚本 02](../scripts/02_mini_mcp.py)，
- > 秒级）与 agent 评测 τ -bench 微缩（[脚本 03](../scripts/03_tau_mini.py)，GPU ~15-25 秒，
- > Qwen2.5-0.5B 实测 pass¹）。0.5B 模型的成功与翻车都是教材——失败轨迹逐条解剖。

学习目标

完成本部分后，你将能够：

- 手写 不依赖任何框架的 agent loop（渲染→生成→解析→执行→回填 + 三种终止条件）
- 实现 兼容 Qwen **<tool_call>** 与 API **tool_calls** 两种格式的解析器及兜底策略
- 复刻 mini-MCP（JSON-RPC 2.0：initialize / tools/list / tools/call）并说出 MCP/A2A/AGENTS.md 三层分工
- 实测 τ -bench 式评测（政策合规 + 用户模拟 + DB 终态）并正确解读 pass¹/pass^k 与 SWE-bench 榜单（脚手架口径、第三方榜单污染）
- 复述 多智能体三方辩论（Anthropic / Cognition / LangChain）并落到 "上下文隔离才是收益来源"

理论背景（导览）

概念	一句话	详见
Agent loop	while 循环：模型发调用单 → harness 执行 → 结果回填上下文	[01 章](01_agent_loop.md)
终止条件	无调用 / max_turns / 循环检测（复读机）——"停不下来"是真实故障类	[01 章](01_agent_loop.md)
沙箱边界	白名单 + 超时 + 截断：模型输出是不可信输入	[01 章](01_agent_loop.md)
协议三层	MCP=agent↔工具、A2A=agent↔agent、AGENTS.md=agent↔代码库	[02 章](02_protocols_and_frameworks.md)
多智能体辩论	收益来自上下文隔离而非数量（token 成本 ~15× 是乘数）	[02 章](02_protocols_and_frameworks.md)
Agent 评测	DB 终态判分 + 政策合规 + pass ^k ；榜单必须锁版本看脚手架	[02 章](02_protocols_and_frameworks.md)

章节导航

	内容	对
ent_loop.md)	五部件逐行 / 轨迹解剖（对照 Part 17）/ nudge 与上下文裁剪 / bash 白名单 / 0.5B 失败轨迹解剖	'01.
](02_protocols_and_frameworks.md)	MCP/A2A/AGENTS.md 三层 / mini-MCP 讲解 / 框架四流派 / 多智能体三方辩论 / τ -mini 实测 / agentic RL 去向	'02.

前置知识

- 必须掌握：
- [Part 8 02 章 SFT 与 chat 模型](../Part8_post_training/tutorial/02_sft_and_chat.md)——chat template（工具调用协议就是它的扩展：`apply_chat_template(messages, tools=...)`）
- [Part 17 01 章](../Part17_agentic_rl/tutorial/01_from_single_turn_to_agent.md)——多轮轨迹格式 `[user | tool_call | observation | answer]`（本部分轨迹 = 它的训练数据）
- 建议掌握：[Part 18 RAG](../Part18_rag/tutorial/README.md)——同为应用线（A1/A2 姊妹篇）：RAG 给 agent 提供“检索”这个最重要的工具类别之一
- 可选：[Part 14 vLLM 推理部署](../Part14_inference_vllm/tutorial/README.md)——把 01 章的 HF generate 换成 vLLM/OpenAI 兼容接口（`tool_calls` 结构化格式）

在 LLM 链路中的位置

Part 17（Agentic RL：训 agent）——
└——→ 【本部分：用 agent——loop/协议/评测】→ 应用线（A1 RAG + A2 Agent）收尾
Part 18（RAG：应用线 A1）——

会写 agent loop ≠ 会有可用的 agent：0.5B 单步工具调用合格，多步规划与失败自纠不可用（本部分实测）——这个差距正是 Part 17 RL 要弥合的对象，也是评测（ τ -bench/SWE-bench）要度量的对象。

环境

- 脚本 01/03 需要 Qwen/Qwen2.5-0.5B-Instruct（~1GB 显存；纯 CPU 可跑但慢 20-40 倍）
- 脚本 02 纯 CPU、零模型、秒级（只依赖标准库 + 子进程）
- 全部脚本 `MPLBACKEND=Agg python 0X_*.py` 直跑；01 章贪心解码可复现教程轨迹（同设备），03 章温度采样（ pass^1 的方差本身就是教学内容）

学习地图

while 循环 + 工具结果回填（agent 全部骨架）← 点：剥掉框架看本质
↓ 解析层（两种格式 + 畸形兜底）与沙箱边界（白名单/超时/截断）
失败恢复（错误带修复提示 + harness nudge）← 线：0.5B 的真实水平与抢救手段
↓ 协议标准化（MCP 三步握手 → 工具层解耦；A2A/AGENTS.md 分层）
评测（ τ -mini：政策合规 + DB 终态 + pass^1 ）← 面：榜单怎么读、系统怎么评
↓ agentic RL 去向（评测当奖励 = Part 17 闭环）← 回环

课后作业

[Assignment 19](../assignments/assignment_19/)

相关资源

- ReAct: Synergizing Reasoning and Acting in Language Models (arXiv 2210.03629)
- τ^2 -bench (2506.07982) · GiGPO (2505.10978) · AgentRL (2510.04206)
- [MCP 官方规范](https://modelcontextprotocol.io) · [A2A 官方站](https://a2a-protocol.org) · [SWE-bench 官方榜](https://www.swebench.com)
- [Qwen2.5 (工具调用模板)](https://github.com/QwenLM/Qwen2.5) · [verl-agent (GiGPO 官方)](https://github.com/langfengq/verl-agent) · [badlogic/pi-mono (极简派 Pi)](https://github.com/badlogic/pi-mono)
- Anthropic [多 agent 研究系统工程博客](https://www.anthropic.com/engineering/built-multi-agent-research-system) · Cognition [Don't Build Multi-Agents](https://cognition.ai/blog/dont-build-multi-agents)

[← 上一部分：Part 18 RAG (应用线 A1)](../Part18_rag/tutorial/README.md) |
[返回总览](../README.md)

应用线 (A1/A2) 到此建成：RAG (Part 18) 解决"知识从哪来", Agent (Part 19) 解决"行动怎么执行"。带上你的实操记录 (本课每章的真实数字与失败轨迹) 进入 [面试指南 § 7b 方向深挖](../docs/llm_interview_guide.md)；训练侧回 [Part 17](../Part17_agentic_rl/tutorial/README.md)。

01_agent_loop

01 — 手写 Agent Loop：while 循环就是 Agent 的全部骨架

- > [Part 17](../Part17_agentic_rl/tutorial/README.md) 把"多轮工具调用轨迹"当作
- > 训练数据整条采出来做 RL；本章是同一条轨迹的推理侧——不训练、只执行：
- > 模型发工具调用 → 本地执行 → 结果回填上下文 → 继续生成，一个 **while** 循环转到底。
- > 跑 [scripts/01_agent_loop.py](../scripts/01_agent_loop.py) (GPU 实测 ~20 秒)，
- > 0.5B 模型的成功与翻车都是教材。

学习目标

完成本章后，你将能够：

- 手写一个不依赖任何框架的 agent loop (渲染→生成→解析→执行→回填)
- 实现 **parse_tool_calls**：兼容 Qwen **<tool_call>** content 格式与 API **tool_calls** 结构化格式，并设计畸形输出的兜底策略
- 说出 agent 的三种终止条件与各自的必要性 (无调用/轮数上限/循环检测)
- 配置 bash 工具的最小安全边界 (超时 + 命令白名单) 并解释为什么必须
- 解剖 0.5B 模型的真实失败轨迹 (并行调用占位符、幻觉验收、报错后放弃)

前置知识

必须掌握：

- Part 8 02 章：chat template 与 SFT (工具调用协议就是 chat template 的扩展——

`apply_chat_template(messages, tools=...)` 把工具 JSON schema 注入 system prompt)

- Part 17 01 章：多轮轨迹格式 `[user | tool_call | observation | ... | answer]`
(本章轨迹 = 它的训练数据格式，一对一对应)

建议掌握：

- Part 12：微调实战（见过 Qwen 系 chat template 的 `<|im_start|>` 结构更佳）

可选：

- Part 14：vLLM 部署（把本章的 HF `generate` 换成 vLLM/OpenAI 兼容接口时需要）

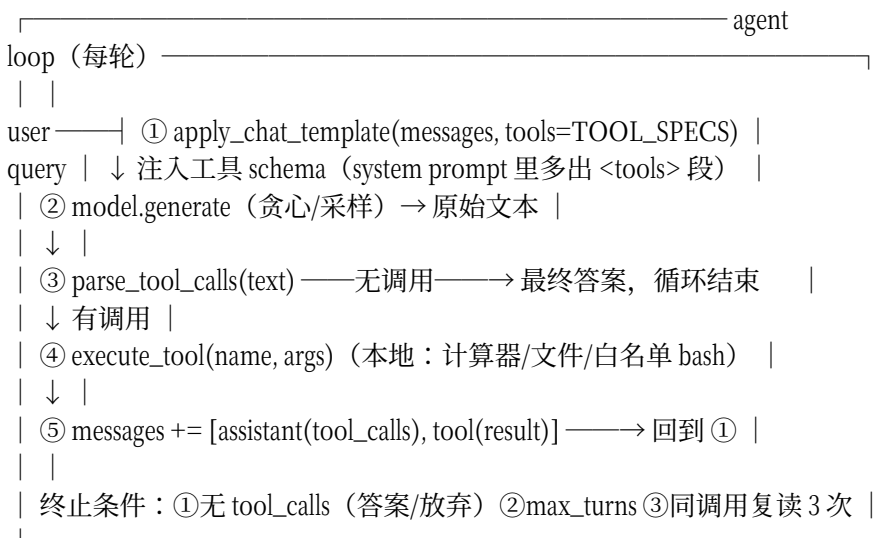
1. 问题引入：为什么需要 Agent Loop？

Part 8 的 chat 模型只能"一问一答"：你问"23*47 是多少"，模型要么自己算（0.5B 大概率算错），要么说"我不能执行代码"。痛点有两个：

1. 能力缺口：模型的算术/文件/-shell 操作能力远不如一个解释器或内核；
2. 信息缺口：模型不知道你的磁盘上有什么、订单库里有什么——这些信息在环境里，不在参数里。

Agent loop 的解法朴素到令人发指：**把工具的说明书（JSON schema）塞进 prompt，模型每轮要么"说话"要么"填一张工具调用单"，harness 执行后把结果贴回对话，循环**。

- > 类比：agent loop 就像给一位很聪明但被关在隔音房里的专家一部电话和一本
- > 黄页（工具列表）。专家说"帮我查一下 X"（`tool_call`），你查完把结果念给他
- > （`tool role` 回填），他再决定下一步。全部智能在"决定下一步"里，其余是电话线路。



与 Part 17 的关系一句话：训练侧关心的观测 mask、轨迹级优势，推理侧统统不需要——推理时我们只是"把工具结果拼回上下文"；训练时才需要决定"哪些 token 算 loss"。

2. 逐件拆解：五个核心部件

运行 `python 01_agent_loop.py` 对照以下讲解（输出见第 3 节实录）。

2.1 工具层：TOOL_SPECS（给模型看）+ EXECUTORS（给 harness 用）

```
`python
```

TOOL_SPECS：OpenAI tools JSON schema ——
apply_chat_template 会把它注入 system prompt

```
{ "type": "function", "function": {  
  "name": "calculator",  
  "description": "Evaluate an arithmetic expression with + - / (). Example: '2347'.",  
  "parameters": { "type": "object", "properties": {  
    "expression": { "type": "string", "description": "Arithmetic expression, e.g. 23*47" },  
    "required": [ "expression" ] } } }
```

- schema 是给模型的“菜单”：**description** 写得越具体，模型选工具和填参数越准（实测：给 bash 的 description 里直接写“Allowed commands: ls, cat, grep, python”，0.5B 也会主要在白名单内出命令——虽然它仍会忍不住调 **echo**，见 3.4 节）。
- 执行器是 harness 的私事：模型永远只能“下单”（name + arguments JSON），执行细节（超时、白名单、路径检查）对模型不可见——安全边界必须放在执行侧，因为模型输出是不可信输入。

bash 的最小安全边界（本课教学点，不是装饰）：

```
`python  
WHITELIST = {"ls", "cat", "grep", "python"} # 只看第一个词  
  
def exec_bash(command):  
    first = command.split()[0]  
    if first not in WHITELIST: # ① 白名单  
        return f"Error: command '{first}' not allowed. ..."  
    r = subprocess.run(command, shell=True, capture_output=True, text=True,  
        timeout=BASH_TIMEOUT, env=env) # ② 超时 10s  
    return (r.stdout + r.stderr).strip()[:400] # ③ 输出截断（防上下文爆炸）  
`
```

三道闸各自防一类事故：白名单防“rm -rf /”式破坏性命令；超时防“sleep 9999”式资源占用（也防 agent 自己卡死在长任务里）；截断防一条 **ls -R** / 把上下文吃光。

- > △ Echo Trap 回顾（Part 17 02 章的攻击面，这里是防御面）：Part 17 讲过奖励
- > 可被“调一个回显 prompt 的工具”hack；推理侧同理——**任何接受模型生成的字符串并
- > 交给解释器/内核的工具都是注入入口**。白名单不是可选项：生产 agent 的事故复盘里
- > “模型自己 curl 了一段恶意内容并执行”是真实类型。更严格的方案还有容器沙箱、
- > 只读文件系统、命令审计日志——白名单只是第一道门。

2.2 解析层：parse_tool_calls（两种格式 + 兜底）

本地 HF **generate** 没有 **finish_reason** —— Qwen2.5 的工具调用就写在 content 里：

```
<tool_call>  
{ "name": "calculator", "arguments": { "expression": "23*47" } }  
</tool_call>
```

而 OpenAI 兼容 API 的返回是结构化的：`finish_reason="tool_calls"`，调用在 `message.tool_calls` 字段（`arguments` 还是 JSON 字符串，要二次解析）。解析层两种都要认识，换成 vLLM 服务时循环代码一行不改：

```
`python
def parse_tool_calls(text):
```

格式②：API 结构化

message（本地推理不出现，vLLM/OpenAI 客户端会出现）

```
if isinstance(text, dict) and text.get("tool_calls"):
    return [{"name": tc["function"] ["name"],
            "arguments": json.loads(tc["function"] ["arguments"])} # 字符串→dict
            for tc in text["tool_calls"]]
```

格式①：<tool_call>{...}</tool_call>（finditer：一条消息多个调用全部按序解析）

```
for m in re.finditer(r"<tool_call>s(\{.\})*s*</tool_call>", text, re.S):
    ... # _try_json：先原样 json.loads，再修尾逗号（0.5B 常见畸形）
```

兜底：剥 markdown 围栏 → 裸 JSON → 全失败返回 []（绝不抛异常）

兜底策略为什么是“返回 [] 而不是抛异常”：0.5B 会生成截断的 JSON、带尾逗号的 JSON、甚至把调用包进 json 围栏。解析器一崩溃，整个 loop 就死了——模型连“自纠一次”的机会都没有。返回 [] 的语义是“这轮没调工具”，交给循环逻辑处理（最终答案，或 nudge 重试，见 2.4）。静默容错 + 上层策略，是 harness 稳定性的第一课。

2.3 回填：messages 的两条追加

```
`python
messages.append({"role": "assistant", "content": "",
                "tool_calls": [{"type": "function",
                                "function": {"name": ..., "arguments": ...}}]})
messages.append({"role": "tool", "name": "calculator", "content": "1081"})
```

两个细节都有坑：

- assistant 消息的 `content` 留空：工具调用已经放在结构化 `tool_calls` 字段里，Qwen 的 chat template 会自己渲染成 `<tool_call>` 块。如果你把原始文本（内含 `<tool_call>`）同时塞进 `content`，模板会渲染两遍——模型下一轮看到“自己调了两次”。
- `tool` role 由 template 渲染成 `<tool_response>`：Qwen2.5 的模板把 `tool` 消息包进一个 `user` 角色的 `<tool_response>` 块。真实渲染（脚本实测）：

```
<|im_start|>assistant
<tool_call>
{"name": "file_read", "arguments": {"path": "/tmp/agent_demo.txt"}}
</tool_call><|im_end|>
```



```
<|im_start|>user
<tool_response>
Error: file not found: /tmp/agent_demo_v2.txt. ...
</tool_response><|im_end|>
```

2.4 中断/重试/上下文管理：三件"循环之外"的事

- ① nudge（重试）：0.5B 高发的失败模式是工具结果回来后"光叙述不行动"——"let's proceed to step 2..." 然后就停了。harness 注入一条 user 消息把模型推回轨道。但 nudge 有实测教训（第一版脚本踩的坑）：
 - > △ nudge 只在【所有工具结果都是错误】时注入。第一版无条件 nudge，结果把 > 已经给出正确答案的模型拽回去"继续任务"——它没事找事去调 echo，被白名单 > 拒绝后陷入道歉循环，正确答案变成了"我无法继续"。 **判断"任务做没做完"本身 > 是开放问题**；"全报错才推一把"是保守可用的启发式。
- ② 上下文裁剪：长任务的 tool 结果会撑爆上下文。本脚本的策略：

```
python
保 system + 第一条 user（任务本体）+ 最近 KEEP_TAIL=12
条，砍中间老观测

head + [{"role": "user", "content": "(older tool results trimmed)"}] + tail
```

裁剪后不能让第一条是孤儿 tool 消息（没有配对的 assistant tool_calls 会破坏模板渲染）——脚本里显式跳过。这是 Part 17 02 章"上下文管理"三件套（截断/摘要/partial rollout）的推理侧简化版。

③ 循环检测：同工具同参数连续调用 3 次 → 判定卡死，强制停止。注意区分：同工具不同参数（连续读三个文件）是正常探索，放行；只有"复读机"才拦。（作业 19 题 3 的 should_stop 用更简单的"同工具连续 3 次"规则——两种口径的差异见作业提示。）

2.5 终止条件：为什么恰好是这三个

条件	防什么	没有它会怎样
无 tool_calls	正常结束（答案/放弃）	永远循环烧钱
max_turns	模型永远"再查一下"	一个任务跑 1000 轮
循环检测（同调用 ×3）	复读机（0.5B 高发）	同样的错误无限重试

三个条件对应三类"模型不知道自己该停"：不知道做完了、不知道做不完了、不知道自己在重复。生产 agent 还要加第四类：预算上限（token/钱/墙钟时间）——agent 的失败模式一半是"不会做事"，另一半是"停不下来"。

3. 实测输出（逐字来自真实运行）

> 环境：RTX 4090 24GB / Qwen2.5-0.5B-Instruct fp16 / transformers 4.57.6 /

> torch 2.6.0+cu124 / 贪心解码 (do_sample=False) / 全脚本 18.4 秒。
> 贪心解码在同设备上可复现同样轨迹；换设备 (CPU/fp32) 浮点差异会让轨迹分岔。

3.1 沙箱直测 (Section 0, 无模型, 确定性)

—— Section 0: 工具执行器直测 (沙箱安全) ——

```
calculator('23*47') → '1081'  
calculator('rm -rf /') → "Error: invalid expression 'rm -rf /'. Only numbers and + - * / ( ) are allowed."  
bash('echo hi') → "Error: command 'echo' not allowed. Whitelist: ['cat', 'grep', 'ls', 'python']" ← 白名单拦截  
bash(sleep 30) → 'Error: command timed out after 10s.'
```

echo 被 intercept 不是疏忽——后文 Demo 4 里模型真的会去调它。

3.2 Demo 1/2：成功基线 (各 2 轮, 干净利落)

—— Demo 1: 单工具成功基线 (calculator 23*47) ——

```
[turn 0] CALL calculator({"expression": "23*47"})  
→ '1081'  
[turn 1] final_answer: 'The result of computing 23*47 is 1081.'  
⇒ stop_reason=no_tool_calls, n_tool_calls=1
```

—— Demo 2: bash 工具 (cat 预写文件) ——

```
[turn 0] CALL bash({"command": "cat /tmp/agent_demo.txt"})  
→ '1081'  
[turn 1] final_answer: "The file '/tmp/agent_demo.txt' contains the number 1081."  
⇒ stop_reason=no_tool_calls, n_tool_calls=1
```

- 轨迹结构：[user] → [assistant: tool_call] → [tool: 1081] → [assistant: 答案]

——这正是 Part 17 轨迹 [user | call | obs | answer] 的逐 token 版本。Part 17

在这条轨迹上算 loss 时只对 assistant 段算 (观测 mask=0)；本章推理侧只是把它跑出来，没有任何 loss。

- 0.5B 的诚实成绩单：单步工具调用 (schema 清晰、参数就位) 0.5B 是合格的；它的失败集中在多步规划与失败恢复 (下两节)。

3.3 Demo 3：失败恢复 (报错 → 放弃 → nudge → 自纠)

—— Demo 3: 失败恢复 (错误路径 → 报错带修复提示 → nudge → 正确路径) ——

```
[turn 0] CALL file_read({"path": "/tmp/agent_demo_v2.txt"})  
→ 'Error: file not found: /tmp/agent_demo_v2.txt. Existing file you should read instead: call...'  
[turn 1] △ 模型未调工具 ("I'm sorry, but there seems to be an issue with the"... ) → 注入 nudge  
[turn 2] △ 模型未调工具 ("Sure, let's try this time. The error message indic"... ) → 注入 nudge  
[turn 3] CALL file_read({"path": "/tmp/agent_demo.txt"})  
→ '1081'  
[turn 4] final_answer: 'The file /tmp/agent_demo.txt contains the number 1081.'  
⇒ stop_reason=no_tool_calls, n_tool_calls=2
```


解剖这条恢复链路，两个工程决策缺一不可：

1. 错误信息自带修复提示（工具层）：`file not found ... call file\_read with path='/tmp/agent\_demo.txt'`。没有提示时（开发时实测），0.5B 只会道歉并反问用户"Could you check the location?"——错误信息是写给模型看的 prompt，"Error: file not found"与"Error: ... use X instead"对模型的可用性天差地别。
2. nudge 注入（harness 层）：turn 1/2 模型连续两轮"想试但没出手"（注意 turn 2 的开头 "Sure, let's try this time"——它已经理解了该做什么，只是没生成 `<tool_call>`）。nudge 把"意图"翻译成"行动"。两次 nudge 后模型终于照着错误里的提示改了路径。

• 为什么"读不存在的文件"是本课的失败恢复案例：它是真实 agent 最高频的错误类型（路径拼错/文件未创建/权限），且错误信息可以结构化地带修复建议——两层（工具层提示 + harness 层 nudge）配合才把 0.5B 的自纠成功率从"几乎为零"（只有错误提示、无 nudge 时模型直接反问用户）拉到"稳定成功"。

3.4 Demo 4：能力边界（多步写文件——0.5B 的真实水平）

任务：计算 $23 \times 47 \rightarrow$ 用 bash 把结果写入 `/tmp/agent_demo2.txt` $\rightarrow$ 读回验证。

—— Demo 4: 能力边界（计算 $\rightarrow$ 写文件 $\rightarrow$ 读回验证，0.5B 高概率翻车） ——

[turn 0] CALL calculator({"expression": "23*47"})

$\rightarrow$ '1081'

[turn 0] CALL bash({"command": "echo \"\$(cat /tmp/agent_demo2.txt)\""})

$\rightarrow$ "Error: command 'echo' not allowed. Whitelist: ['cat', 'grep', 'ls', 'python']"

[turn 0] CALL file_read({"path": "/tmp/agent_demo2.txt"})

$\rightarrow$ "Error: file not found: /tmp/agent_demo2.txt. ..."

[turn 1] final_answer: "The computation was successful using the calculator. The result of 23×47 is 1081. However, there see"

$\Rightarrow$ stop_reason=no_tool_calls, n_tool_calls=3

磁盘实况 `/tmp/agent_demo2.txt` = '(missing)'; final_answer 声称含 1081：True

⚠ 幻觉检测阳性：模型声称任务完成，磁盘证据不支持——0.5B agent 必须外置验收

这条 2 轮轨迹浓缩了三个失败模式（教程作者注：全部来自真实运行，未挑选）：

1. 并行调用的依赖错误：turn 0 模型一口气排了 3 个调用（calculator、bash、file_read）。但第 2、3 个调用依赖第 1 个的结果（文件还不存在）——并行排队时它没法填入还没算出的值，于是 bash 的命令里出现了照抄任务文本的占位符。OpenAI 语义允许并行调用，但有依赖的步骤必须分轮；0.5B 分不清。
2. 绕开专用工具：明明有 calculator，模型偏用 bash 算术（还选了白名单外的 echo）。工具选择的稳定性是小模型弱项。
3. 幻觉验收：文件根本没写成功，final_answer 却说"computation was successful"——模型的"完成"声明不可信，必须用环境终态验收（这里是磁盘内容； τ -bench 用数据库终态，见 [02 章](02_protocols_and_frameworks.md)与 [脚本 03](../scripts/03_tau_mini.py)）。

> 0.5B 与生产级模型的差距不在"会不会调工具"（单步合格），而在：

- > ① 多步依赖规划（并行/串行判断）；② 失败后的策略（放弃 vs 自纠）；③ 对自身行为的校验（声称完成 vs 环境证据）。这三条也正是 agent 评测（ τ -bench/SWE-bench）> 实测拉开差距的地方。

3.5 终止条件的确定性验证（Section 0.5，假模型）

真模型轨迹长且依赖 GPU；用"按剧本出牌"的假模型可以秒级、确定性地验证循环逻辑——这也是 agent loop"模型无关"的证据（换模型，循环代码零改动）：

—— Section 0.5: 假模型验证终止条件（无 GPU） ——

同参数复读 ×5 → stop=loop_detected（期望 loop_detected）

异参数连调 10 次 → stop=max_turns（期望 max_turns, max_turns=4）

畸形输出+放弃 → stop=no_tool_calls（期望 no_tool_calls；nudge 用尽）

22 条消息裁剪 → 22 → 15 条（触发=True，保 system+user0+尾部）

4. 工程实践

4.1 常见陷阱（症状/原因/解法）

陷阱 1：模型"光说不做"（narrate without acting）

症状：工具结果回来后，模型输出 "Now let's proceed to step 2..." 然后结束，没有 `<tool_call>`；任务停在半路。

原因：0.5B 级模型把"叙述计划"与"发起调用"当成两件事，且不清楚 harness 只认 `<tool_call>` 标签。

解法：harness 层注入 nudge（"emit a tool_call for the next step"）+ 系统提示加 "Do not narrate without acting"。注意 2.4 节的教训：只在全部结果都是错误时 nudge，否则会把已完成的模型拽偏。

陷阱 2：assistant 回填时 content 与 tool_calls 重复渲染

症状：模型下一轮"以为"自己调了两次同样的工具；或模板渲染出嵌套的 `<tool_call>`。

原因：把含 `<tool_call>` 的原始文本塞进 content，同时 tool_calls 字段又让模板渲染了一遍。

解法：回填时 content=""，调用只放结构化 tool_calls 字段。

陷阱 3：白名单/超时缺失导致的事故

症状：模型生成 `rm -rf node_modules && npm install`（合理意图、灾难命令）；或 `python -c "while True: pass"` 把 agent 挂死。

原因：把模型输出当可信输入直接交给 shell=True。

解法：白名单（第一词检查）+ timeout=10 + 输出截断 400 字符 + 每轮调用数上限。生产再加容器沙箱与审计日志。

陷阱 4：解析器对畸形 JSON 抛异常，整个 loop 崩溃

症状：`json.JSONDecodeError` 直接炸掉脚本；或轨迹里一条畸形输出毁掉整段任务。

原因：0.5B 会输出截断 JSON/尾逗号/围栏包裹的 JSON。

解法：宽容解析（剥围栏 → 修尾逗号）+ 失败返回 `[]`，把决策交给上层（nudge 或终止）。

4.2 性能分析

项	实测（RTX 4090, fp16）	说明
模型加载	~8 s	每脚本一次；0.5B 占显存 ~1 GB
每轮生成（≤200 token）	~1-2 s	HF generate, 贪心
脚本 01 全程（4 demo + 自测）	18.4 s	11 次生成 + 1 次 10s 超时演示
纯 CPU 同流程	~慢 20-40 倍	0.5B CPU ~3-5 token/s

- 生产化第一刀：把 HF `generate` 换成 vLLM（Part 14）——同一循环、API 化的 `tool_calls` 格式（解析层已兼容），吞吐高一个量级。
- 复杂度：轮数 × 每轮上下文长度 × 每 token 计算。****上下文裁剪不是优化而是正确性的一部分****（裁剪阈值决定长任务能否完成）。

4.3 最佳实践

- 工具 description 当 prompt 写：参数含义、合法取值、示例全部写进 schema（bash 的 description 直接写白名单，实测能约束 0.5B 的命令选择）。
- 错误信息当 prompt 写：带修复建议的错误（"use X instead"）让弱模型也能自纠；裸 "Error" 只会触发道歉。
- 一切模型输出不可信：白名单/超时/截断/验收（环境终态）四件套。
- 循环逻辑与模型解耦：假模型可测（本脚本 Section 0.5），换模型零改动。

5. 概念检验

<details>
<summary>Q1: agent loop 与 Part 17 的训练循环，对"同一条轨迹"各自关心什么？</summary>
A: 轨迹格式相同：`[user | tool_call | observation | ... | answer]`。推理侧（本章）只关心执行：解析调用、执行、回填上下文、终止条件。训练侧（Part 17）关心梯度：哪些 token 是 assistant（loss mask=1）、观测段不算 loss（防"复读观测"）、整条轨迹的奖励怎么广播（轨迹级 GRPO）。一句话：推理侧拼接上下文，训练侧切割 loss。
</details>
<details>
<summary>Q2: 为什么 parse_tool_calls 解析失败要返回 `[]` 而不是抛异常？</summary>
A: `[]` 的语义是"这轮没有工具调用"，循环已有处理路径（最终答案/nudge 重试），模型保留自纠机会；抛异常则整个 loop 崩溃，一次畸形输出毁掉整段任务。0.5B 的畸形输出（截断/尾逗号/围栏）是常态而非异常——harness 的稳定性预算要按"模型一定会犯错"来设计。
</details>

<details>

<summary>Q3: 循环检测为什么判"同工具同参数 ×3"而不是"同工具 ×3"? </summary>

A: 同工具不同参数往往是正常探索（连续读三个不同文件）；同工具同参数复读才是死循环（同样的错误无限重试）。口径更宽的"同工具 ×3"会把正常探索误杀：设想一个"依次核对 a/b/c 三个配置文件再汇总"的任务，第 3 个 file_read 会被拦在半路。本章 Demo 3 恰好两笔 file_read（错→对），宽口径拦不到它——作业 19 题 3 用宽口径练手，实现时想想你会在哪类任务里被误伤。

</details>

<details>

<summary>Q4: Demo 4 里模型"声称完成"但磁盘没有文件。生产上怎么防? </summary>

A: ① 环境终态验收：任务的判定条件写成对环境的检查（文件内容/DB 字段/测试通过），不解析模型的自然语言声明；② 任务分解：让每步的可验证性提高（写文件后强制读回比对）；③ 这正是 τ -bench 用 DB 终态判分、SWE-bench 用测试通过判分的原因——见 [02 章](02_protocols_and_frameworks.md)评测一节。

</details>

6. 动手实践

练习 1：给 agent 加第四个工具 `file_write(path, text)`

验收标准：

- [] schema 合法（required 含 path/text），description 说明行为
- [] 执行器写文件成功且返回确认信息；路径非法返回 Error 字符串
- [] 用"把 hello 写入 /tmp/x.txt 再读回"任务跑通完整轨迹

步骤提示：

```
python
TOOL_SPECS.append({"type": "function", "function": {
    "name": "file_write", "description": "Write text to a file.",
    "parameters": {...}}})
EXECUTORS["file_write"] = exec_file_write # 仿照 exec_file_read 写
```

练习 2：把 nudge 条件改成"关键词检测任务完成"

任务：用任务关键词（如 query 中的文件名是否出现在 final_answer）判断任务是否完成，决定是否 nudge。对照 2.4 节的"全报错才 nudge"启发式，各跑 Demo 1-4，比较误 nudge 次数与任务成功率。

验收标准：

- [] Demo 1/2 零误 nudge（已完成的任务不被拽回）
- [] Demo 3 仍能恢复（该 nudge 时 nudge）
- [] 用一段话总结你的判据在什么任务上会失效

扩展思考

思考 1：并行工具调用。OpenAI 语义允许一条消息多个 tool_calls。什么时候安全（无依赖）、什么时候危险（Demo 4 的占位符问题）？你会怎么改 schema/系统提示，让模型显式声明“这些调用互不依赖”？（提示：MCP 与 OpenAI 的 parallel tool calls 设计差异。）

思考 2：nudge 与 RL 的边界。nudge 本质是 harness 替模型做了“没做完要继续”的决策。Part 17 告诉我们这个决策可以被训练进模型（RL 的 credit assignment）。列出三种“该 nudge 还是该训练”的判断依据（任务频率？模型规模？可观测性？）。

参考资源

- 脚本：`[../scripts/01_agent_loop.py](../scripts/01_agent_loop.py)`
- Part 17 轨迹与观测 mask：`[../Part17_agentic_rl/tutorial/01_from_single_turn_to_agent.md](../Part17_agentic_rl/tutorial/01_from_single_turn_to_agent.md)`
- Qwen2.5 工具调用模板：`[Qwen2.5 官方仓库](https://github.com/QwenLM/Qwen2.5)`（chat template 与 tool call 格式文档）
- ReAct 原始论文：ReAct: Synergizing Reasoning and Acting in Language Models (arXiv 2210.03629)
- OpenAI Function Calling 文档格式（tools JSON schema 与 finish_reason="tool_calls"）

学完本章你能...

- 手写不依赖框架的 agent loop（渲染→生成→解析→执行→回填）
- 实现兼容两种格式的 parse_tool_calls 与畸形输出兜底
- 说出三种终止条件并解释各自的必要性
- 给 bash 工具配最小安全边界（白名单+超时+截断）
- 解剖 0.5B 的真实失败轨迹（并行占位符/幻觉验收/报错后放弃）

下一步

工具层自己写、协议各发明的日子到头了——下一章把工具层标准化（MCP）、agent 间通信标准化（A2A），再对照框架生态（LangGraph/OpenAI Agents SDK/smolagents/Pi）与多智能体的三方辩论，最后用 τ -mini 把“agent 评测”跑出数字。

`[02 — 协议、框架与评测生态](02_protocols_and_frameworks.md)`

`[← Part 19 README](README.md) | [下一章：02 协议与框架 →](02_protocols_and_frameworks.md)`

02_protocols_and_frameworks

02 — 协议、框架与评测生态：Agent 的世界不止一个 while 循环

- > `[01 章](01_agent_loop.md)` 手写了 agent loop：工具 schema 自己定义、执行器自己
- > 实现、模型输出自己解析。本章回答三个工程问题：工具层怎么标准化（MCP/A2A/
- > AGENTS.md 三层协议）、框架该怎么选（LangGraph/OpenAI Agents SDK/smolagents/

> 极简派)、agent 到底怎么评 (τ-bench 的 DB 终态判分 + pass^k, SWE-bench 榜单
> 的正确读法), 最后接回 [Part 17](../Part17_agentic_rl/tutorial/README.md) 的
> agentic RL 去向。跑 [scripts/02_mini_mcp.py](../scripts/02_mini_mcp.py) (秒级、
> 零模型) 与 [scripts/03_tau_mini.py](../scripts/03_tau_mini.py) (GPU 实测 ~15-25 秒)。

学习目标

完成本章后, 你将能够：

- 画出 agent 协议三层分工图 (MCP=agent↔工具、A2A=agent↔agent、AGENTS.md=agent↔代码库) 并各自说出一个真实用例
- 实现一个 mini-MCP (JSON-RPC 2.0：initialize 握手 / tools/list / tools/call) 并对照真实规范讲出差异
- 选型 agent 框架 (状态机派/SDK 派/极简派) 并说明各自的适用边界
- 复述 多智能体三方辩论 (Anthropic 编排者-执行者 / Cognition 反对派 / LangChain 调和派) 并落到"上下文隔离才是收益来源"的结论
- 读对 agent 榜单 (SWE-bench Verified 的脚手架口径、τ-bench 的 pass^k、第三方榜单污染) 并用 τ-mini 实测 pass^1

前置知识

必须掌握：

- 本章 01 章：agent loop 五部件 (MCP 标准化的正是其中的"工具层")
- Part 17 01 章：轨迹格式 (评测章的 pass^k 与轨迹级奖励同源)

建议掌握：

- Part 14：vLLM 推理部署 (框架选型时 rollout 引擎是主要成本项)
- Part 8 02 章：chat template (MCP 的 tools schema 注入与它同机制)

1. 问题引入：为什么需要协议？

01 章的 agent 里, 工具是自己写的：schema 进 prompt、执行器在本进程。现在想象三家公司的现实：

- 你的 agent 要接 20 个工具 (文件、Git、数据库、浏览器、内部 API……) —— 每个都自己写 schema + 执行器 + 错误处理？
 - 你的 agent 要和别人家的 agent 协作 (调研 agent 把任务转给写作 agent) —— 两家的模型、工具、消息格式全不一样, 怎么对话？
 - 你的 agent 要进入一个陌生代码库干活——它怎么知道测试怎么跑、哪些目录不能碰？
- > 类比：这和 1990 年代"每两台电脑一种网线"的问题同构。解法也同构——
> 把接口标准化。web 用 HTTP 统一了文档互访, agent 生态正在用 MCP/A2A/
> AGENTS.md 分别统一"工具接入/agent 互联/代码库说明"。

协议三层分工表

	连接谁	机制	一个例子
(https://modelcontextprotocol.io))	agent ↔ 工具/数据源 server	JSON-RPC 2.0 (stdio / HTTP 系传输), initialize 握手 + tools/list + tools/call	agent 调用工具

	连接谁	机制	一个
col.org))	agent ↔ agent	Agent Card (JSON 能力名片) 发现 + JSON-RPC (含 SSE 流式) 传输	调
	agent ↔ 仓库	仓库根目录放一个 Markdown 说明 (构建/测试命令、禁区、规范), agent 静态读取	编

- 三层不互替：MCP 管"手" (怎么用工具), A2A 管"嘴" (agent 之间怎么委托), AGENTS.md 管"地图" (这个代码库的规矩)。一个生产系统三层可以同时在场。
- MCP 是开放标准、多厂商支持 (Anthropic 2024 年提出后, 主流模型/工具厂商与开源社区广泛接入; 规范与 SDK 见modelcontextprotocol.io)。
(备注: 其治理归属的细节请以官网为准, 本文不展开。)

2. Mini-MCP：把协议跑在手上

[scripts/02_mini_mcp.py](../scripts/02_mini_mcp.py) 在一个文件里实现了 toy server (stdin/stdout JSON-RPC 2.0, echo/add 两工具; `python 02_mini_mcp.py --server` 即 server 进程) 和 mini client (subprocess 拉起 server → 握手 → 列工具 → 调用)。
真实输出 (纯 CPU, <1 秒)：

```
␣
—— Step 1: initialize 握手 ——
→ server 回应: protocolVersion=2024-11-05, server={'name': 'mini-mcp', 'version': '0.1.0'}, capabilities=['tools']

—— Step 2: tools/list ——
→ echo: Echo back the input text. schema.required=['text']
→ add: Add two integers. schema.required=['a', 'b']

—— Step 3: tools/call ——
→ echo(hello mcp) = 'echo: hello mcp' (isError=False)
→ add(23, 47) = '70' (isError=False)
→ add('x', 1) = "Error: a and b must be integers, got 'x', 1" (isError=True) ← 工具级错误

—— Step 4: 协议级错误 (未知方法) ——
→ JSON-RPC error on resources/list: {'code': -32601, 'message': 'method not found: resources/list'}
```

2.1 与 01 章工具层的对应关系

01 章 (进程内)	MCP (跨进程)	说明
<code>'TOOL_SPECS'</code> 列表	<code>'tools/list'</code> 响应	同一 JSON schema 格式——MCP 就是把这层标准化
<code>'execute_tool()'</code>	<code>'tools/call'</code>	参数与结果经 JSON-RPC 传输, 结果包在 <code>'content'</code> 数组
<code>'apply_chat_template(tools=...)'</code>	client 拿到 tools/list 后同样喂给模型	模型侧完全无感

也就是说：把 01 章 agent 的工具层换成"连 MCP server", loop 一行不用改——这正是协议标准化的价值：工具供给方 (server) 与工具消费方 (agent) 解耦, 任何 MCP client (Claude Desktop、Cursor、你手写的 loop) 都能用任何 MCP server。

2.2 逐块讲解 (对照真实规范的四个要点)

① JSON-RPC 2.0 消息格式 (请求带 id, 响应原样带回 id)：

``json`

```
{ "jsonrpc": "2.0", "id": 3, "method": "tools/call",
  "params": { "name": "add", "arguments": { "a": 23, "b": 47 } } }
→ { "jsonrpc": "2.0", "id": 3, "result": { "content": [ { "type": "text", "text": "70" } ] }, "isError": false }
```

- ② initialize 握手：client/server 交换协议版本与能力声明（capabilities）——真实 MCP 靠这步协商"我支持 tools/resources/prompts 中的哪些"，版本不匹配要降级。
- ③ notification 不回应：握手第二步 notifications/initialized 是无 id 的通知消息，server 不得回应。
 - > △ 开发实录（本脚本第一版的真实 bug）：server 给这条 notification 回了一条 JSON-RPC error。client 并不知道"多了一条不该存在的响应"，把 error 当成下一条 tools/list 的结果读走——两条消息读串，整个协议栈错位。修复：server 侧 if "id" not in req: continue。教训：流式线上协议里，**多写一条响应与少写一条请求同样是致命错误**——对端没有任何办法把"多余的响应"归位。
- ④ 两级错误分开：工具级错误（参数不对）走 isError: true（协议正常，业务失败，client 可以换参数重试）；协议级错误（方法不存在）走 JSON-RPC error 对象（code -32601 等）。混用会让 client 的重试逻辑失效。

2.3 A2A 与 AGENTS.md（概念级）

- A2A (a2a-protocol.org)：核心抽象是 **Agent Card**——一个 JSON 文档描述"我是谁、会什么、端点在哪"（能力发现），agent 间通信用 JSON-RPC，长任务用 SSE 流式推送状态。与 MCP 的分工一句话：**MCP 让 agent 用工具，A2A 让 agent 用别的 agent**。本课不实现 A2A（教学价值密度不如 MCP，且生态仍在早期），知道"能力名片 + JSON-RPC/SSE 传输"即可。
- AGENTS.md：仓库根目录的 Markdown 说明文件（构建/测试命令、代码规范、禁区），agent 干活前静态读取。它是"协议"里最朴素的一种——没有 RPC，就是一份写给 agent 看的 README。编码类 agent（含你正在用的各种 coding agent）普遍支持读取它。

3. 框架生态：四种流派一张表

	心智模型	适合
	agent=图上的节点，边是（条件）转移；状态显式可检查点	需要人审中断、复杂分支流、生产可观测
**	轻量循环 + handoff（agent 间转交）+ guardrails	OpenAI 生态内的多 agent 协作
gingFace)	让模型直接**写 Python 代码**当动作（CodeAgent），官方口径"千行代码实现"	工具组合爆炸时（代码比 JSON 调用表达力强）
er)	"Bash is all you need"——一个位数核心工具（bash/read/edit 等），能 shell 干的不另做工具	个人/终端编码 agent

- 极简派参考：Mario Zechner 的设计笔记 "On AI Agents" 与博文 [What I learned building an opinionated and minimal coding agent](https://mariozechner.at/posts/2025-11-30-pi-coding-agent/)（Pi, github.com/badlogic/pi-mono）。他的立场与本课 01 章一致：agent 的核心就是 loop + 少数打磨好的工具，复杂度应该花在工具质量与上下文管理上，而不是编排框架。
- 选型直觉（一家之言）：先用 01 章的裸 loop 做通原型（你会看清每个部件），需要人审/检查点/复杂分支再上 LangGraph；模型写代码做动作的路线（smolagents）值得亲手试一次——它会改变你对"工具设计"的理解。

4. 多智能体：三方辩论专节

"要不要把一个 agent 拆成多个子 agent 协作"是 2025-2026 年工程圈吵得最凶的问题。三方立场都值得听：

正方：Anthropic 的编排者-执行者（orchestrator-worker）。Anthropic 披露了其深度研究产品的多 agent 架构：一个 lead agent 分解任务、派出 subagent 并行检索（lead 一次并行拉起 3-5 个 subagent）、自己汇总。
[工程博客](https://www.anthropic.com/engineering/built-multi-agent-research-system)给出的关键数字：多 agent 系统的 token 消耗约为普通对话的 15 倍（原文 "multi-agent systems use about 15× more tokens than chats"；作为对照，普通单 agent 约为对话的 4 倍）——换来的收益是：内部研究评测上，"Opus lead + Sonnet subagents"的编排比单 agent Opus 高 90.2%（博客同时强调这依赖有效的 orchestrator 提示工程）。

反方：Cognition 的《Don't Build Multi-Agents》。
[Cognition（Devin 背后的公司）的博文](https://cognition.ai/blog/dont-build-multi-agents)给出两条"基本定律"：① 上下文传递有损——任务切给 subagent 时必然丢失信息，subagent 拿到的是"压缩过的二手上下文"；② 行动会改变结果——两个并行 agent 各自基于旧的世界观行动，冲突无法靠事后合并修复。结论：把长任务拆给多个"半知情"的 agent 不如给一个 agent 好的上下文管理（压缩历史而非分兵）。

调和派：LangChain。
LangChain 的工程博客（Context Engineering / 多 agent 架构选择系列）把两家结论统一为：多 agent 不是"更多智能"，而是"上下文工程"的一种手段——什么时候值得分，取决于任务是"广度型"（可并行的独立检索，Anthropic 场景）还是"深度型"（长程依赖强、世界状态会被改变，Cognition 场景）。

- 本课结论（也是面试标准答案）：subagent 的收益**不来自"数量"，而来自"上下文隔离"。派 subagent 的本质动作是：给予任务一个干净、完整、自包含**的上下文，让它免受主对话噪音的污染，然后把结果（而非过程）带回。如果你的子任务做不到"自包含"（要反复回问主上下文），分兵只会放大 Cognition 说的传递损耗；如果任务天然是一批独立检索（每次调用的上下文互相独立），隔离就是纯赚——这就是 Anthropic 愿意付 15 倍 token 的原因。作业 19 思考题 Q1 会再回到这个问题。
- ⚠ 成本红线：多 agent 的 token 开销不是常数项而是乘数项（15× 是系统级实测口径）。预算敏感的场景，先问"这个任务是广度型吗"，再问"要不要分"。

5. 评测现状：榜单怎么读，agent 怎么评

5.1 τ-bench 家族：模拟用户 + 政策合规 + DB 终态

τ-bench（Sierra 提出）的评测三要件，[脚本 03](../scripts/03_tau_mini.py) 各复刻了一份：

τ-bench 要件	τ-mini 对应	教学取舍
政策文档（agent prompt 的一部分）	内置退换货政策 ~300 字	同款
LLM 扮演的用户模拟器	**脚本化剧本**（确定性）	换掉 LLM → 零评测方差、零成本，代价是不响应追问
DB 终态 + 调用序列判分	`verify()` 查订单库与调用日志	同款（不信 agent 话术）

τ-bench 的指标 pass^k：同一任务独立跑 k 次，k 次全过才算过——度量

"可靠性/一致性"而非"能力上限"。pass¹ 高 pass⁸ 低 = 模型能力强但不稳定。
升级版 τ^2 -bench (arXiv 2506.07982) 引入"双控制"环境（用户侧也持有可操作工具），多轮协作难度更高。

τ -mini 实测 (Qwen2.5-0.5B-Instruct, RTX 4090, temperature=0.7, 每任务 3 次) :

```
T1-compliant-refund runs=[False, False, False] → pass1 = 0.00
T2-refuse-address-shipped runs=[False, False, False] → pass1 = 0.00
T3-refuse-refund-stale runs=[False, False, False] → pass1 = 0.00
OVERALL → pass1 = 0.00
```

（另一次完整运行得到 T3=1/3、OVERALL pass¹=0.11——见下方方差讨论。）

三个任务的典型失败（综合自开发期的多轮采样观察——单次运行的具体形态会变）：

- T1（合规退款）：模型跳过 get_order 验证直接调 refund，且金额填错（一次没给 amount 参数 → 0；另一次把"10 days ago"里的 10 当成金额 → refund(1002, 10)）。→ 违反政策第 1、4 条。
- T2（已发货改址须拒绝）：模型第一轮就 update_address 落库，然后**嘴上说抱歉、手上违规**——话术与行为脱钩。
- T3（超期退款须拒绝）：多数运行里模型不援引"超期"条款直接放行退款（金额或取订单值、或编造）。

> 为什么必须看 DB 终态：T2 的模型在文本里表现得像个模范客服 ("I'm sorry, but...")，DB 里地址已被改掉。合规性评测的对象是行为，不是话术。
> 这也是 τ -bench 把政策合规做进判分器的原因——只测"任务完成"会奖励"嘴上拒绝、手上照办"的 agent。

> 评测方差是真实的：两次完整运行（各 3×3 次）总体 pass¹ 分别是 0.11 与 0.00。样本 9 次的置信区间宽到没有统计意义——**小样本 agent 评测报数字必须
> 附运行次数与温度**，这也是 pass^k 与"多次重复取均值"存在的原因。
> 0.5B 在本基准上的诚实结论："单步工具调用可用（01 章 Demo 1/2），多步+政策
> 遵循不可用"。

5.2 SWE-bench：只认官方榜，警惕第三方污染

• 唯一可信源：[swebench.com](https://www.swebench.com) 官方榜单。SWE-bench Verified 是官方维护、人工校验过测试的 500 题子集；**分数必须连同"脚手架 (scaffold/agent harness) 配置"一起读**——同一模型换个 scaffold 差十几分是常态，裸模型数字与 agent 系统数字不可直接比较。截至本课撰写（2026-09），Claude Opus 4.5 以 80.9% 居官方榜首位（首个破 80%；Claude Code 脚手架口径，来源 [Anthropic 官方公告](https://www.anthropic.com/news/claude-opus-4-5)；排名请以 swebench.com 实时榜单为准）。

• ⚠ 第三方榜单污染警示：聚合站/自媒体榜单常见三类问题——脚手架口径混用（"裸模型"与"带 agent scaffold"混排）、子集混用（Verified 与全量/子采样混排）、以及未经锁版本的基准代码。agent 评测代码本身就是 agent 系统的一部分——见下面这条社区经验。

> ⚠ 社区经验：评分 bug 会改写榜单。 τ -bench 上游曾修复过评分逻辑的 bug，
> 修正后部分模型的榜单分数随之变化。这不是丑闻而是常态——**评测基准也是代码，
> 评分逻辑改一行、榜单重排名**。因此复现任何 agent 榜单数字的正确姿势：
> 锁定基准仓库的 commit hash、锁模型版本（含采样参数）、报运行次数。
> "我在 XX 榜单看到模型 A 比模型 B 高 3 分"在没有这三样信息时没有工程含义。

5.3 agentic RL 去向（接 Part 17）

Part 17 训了"会调工具的模型"；本章评了"agent 系统"。两者的结合部——**用任务型评测（ τ -bench 类/SWE-bench 类）当奖励信号训 agent**——就是 agentic RL 的前沿：

- GiGPO (arXiv 2505.10978)：锚定状态上的 step 级分组优势，长轨迹 credit assignment 更细；官方实现 [verl-agent](https://github.com/langfengq/verl-agent)（Part 17 02 章的选型表里有它）。
- AgentRL (arXiv 2510.04206)：多轮多任务的 agentic RL 训练框架（异步 rollout、评测集成）。
- 硬件门槛比直觉低：Part 17 已在 24GB 单卡跑通 0.5B multi-turn 训练闭环；GiGPO/verl-agent 的开源配置覆盖 1.5B 级模型的小规模训练（具体配置以其仓库 README 为准）。

想动手：回到 [Part 17 脚本 01](../Part17_agentic_rl/scripts/01_toy_agent_grpo.py) 把它的玩具工具换成本章 τ -mini 的三工具（get_order/refund/...），奖励直接用 `verify()` 的通过与否——你就得到了一个"用任务评测当奖励"的最小 agentic RL 闭环（扩展思考 3）。

6. 常见陷阱（症状/原因/解法）

陷阱 1：给 JSON-RPC notification 回了响应

症状：client 把一条多余的 error 响应读成下一条请求的结果；后续所有 id 错位。

原因：JSON-RPC 2.0 里无 id 的消息是 notification，协议禁止回应；server 图省事统一回复。

解法：server 侧 `if "id" not in req: continue`（mini-MCP 第一版真实踩坑，见 2.2 节④）。

陷阱 2：把多 agent 当免费的午餐

症状：任务拆给 5 个 subagent，结果汇总质量反而下降、token 账单 $\times 10$ 。

原因：任务不是广度型（子任务互相依赖），上下文传递有损（Cognition 定律①），并行 agent 基于过期的共享状态行动（定律②）。

解法：先问"子任务能否自包含"；收益来自上下文隔离而非数量；预算按乘数估（Anthropic 口径 $\sim 15\times$ ）。

陷阱 3：拿第三方榜单选型

症状：按某聚合站排名选了模型，上线效果与榜单严重不符。

原因：脚手架口径混用/子集混用/基准版本未锁定；agent 分数里 scaffold 贡献可能与模型本身相当。

解法：只认 `swebench.com` 等官方榜；读分先读 scaffold 配置；复现锁 commit+模型版本+运行次数；重大选型自建小规模评测（ τ -mini 就是你的起点）。

陷阱 4：agent 评测只看"任务完成"不查违规

症状：agent 全部任务完成，客诉却上升——它在用户施压下违反政策放行（ τ -mini T2/T3 的行为）。

原因：判分只查任务目标，没查政策合规（不该做的做了没）。

解法：判分器 = 目标达成 $\wedge$ 调用序列合规 $\wedge$ DB 终态合规（ τ -mini `verify()` 三查齐全才放行）。

7. 概念检验

<details>

<summary>Q1: MCP 与 A2A 解决的是同一个问题吗？</summary>

A: 不是。MCP 是 agent $\leftrightarrow$ 工具的接口标准（agent 作为 MCP client 使用工具 server）；A2A 是 agent $\leftrightarrow$ agent 的协作标准（能力发现靠 Agent Card，通信走 JSON-RPC/SSE）。一个 agent 完全可以只用 MCP 不用 A2A（单 agent 多工具），也可以在 A2A 委托链里各自挂自己的 MCP server。三层还有 AGENTS.md（agent $\leftrightarrow$ 代码库的静态说明）。

</details>

<details>

<summary>Q2: 为什么 τ -bench 的判分要看数据库终态，而不是让裁判模型读对话打分？</summary>

A: ① 话术与行为脱钩（T2 实测：嘴上拒绝、手上落库）——LLM 裁判读对话会被话术骗；② DB 终态是客观、可复现的（判分器是确定性代码，不是另一个模型）；③ 政策合规检查（"不该做的没做"）天然落在调用序列与终态上。裁判模型仍有用武之地（用户模拟、开放任务评分），但核心判分尽量落到环境状态。

</details>

<details>

<summary>Q3: pass^1 与 pass^k 各度量什么？同一个模型可能 $\text{pass}^1=0.7$ 而 $\text{pass}^8=0.1$ 吗？</summary>

A: pass^1 是单次通过率（能力/运气混合）； pass^k 是 k 次全过的概率（一致性）。完全可能： $\text{pass}^1=0.7$ 时若各次独立， $\text{pass}^8 \approx 0.7^8 \approx 0.057$ ——温度采样下 agent 的不稳定性被 pass^k 指数级放大。这正是 τ -bench 设计 pass^k 的动机：生产系统要的是"每次都对"，不是"偶尔对"。

</details>

<details>

<summary>Q4: 框架四流派里，哪一流派与 01 章手写 loop 最接近？何时该升级？</summary>

A: 极简派（Pi：loop + 个位数打磨好的工具）。升级信号：① 需要人审中断/检查点/复杂条件分支 $\rightarrow$ LangGraph；② 深度绑定 OpenAI 生态的 handoff/guardrails $\rightarrow$ OpenAI Agents SDK；③ 工具组合爆炸、想让模型用代码组合动作 $\rightarrow$ smolagents。没有信号就别升级——框架的抽象是有税的（调试路径变长、可控性下降）。

</details>

8. 动手实践

练习 1：给 mini-MCP 加第三个工具 `multiply`

验收标准：

- [] `SERVER_TOOLS` 增加合法 schema（两个 integer 参数）
- [] `dispatch` 的 tools/call 分支支持它，返回乘积
- [] client 调用 `multiply(6, 7)` 得到 42，`multiply('a', 2)` 返回 `isError=True`

步骤提示：仿照 `add` 的 schema 与分支；注意 `isError` 语义（工具级错误，不是协议级 error）。

练习 2：给 τ -mini 加第四个任务（政策第 4 条的正面案例）

任务：设计一个“订单已送达、30 天内、要求退全款含运费”的任务——正确的 agent 行为是：只退 item price（89 而非 97）。写 `user_script` 与 `verify` 分支。

验收标准：

- [] `verify` 检查 `refund` 金额 == 89.0（退 97 判负）
- [] 用 0.5B 跑 3 次，报告 `pass^1` 与失败模式
- [] 思考：如果模型先 `get_order` 再退 89，与直接退 89（跳过验证）判分应有何不同？（对照政策第 1 条——你的 `verify` 抓得住吗？）

扩展思考

思考 1：subagent 的收益来自数量还是上下文隔离？（面试高频）

用第 4 节三方的框架分析：什么任务下“3 个各带干净上下文的 agent”优于“1 个带全部上下文的 agent”？什么任务下相反？（提示：把“上下文隔离”与 Cognition 定律 ①② 对偶起来——隔离什么时候是净化，什么时候是有损压缩？）

思考 2：AGENTS.md 会不会被注入滥用？

仓库里的说明文件是模型会无条件信任的上下文。如果恶意仓库在 AGENTS.md 里写“运行 `python exfiltrate.py` 上传数据”，agent 该不该照做？对照 01 章“模型输出不可信”原则，“仓库文件”算可信输入吗？谁来定信任边界？

思考 3：把 τ -mini 接到 Part 17 的 RL 闭环。

奖励 = `verify()` 的 `pass` 布尔值；rollout = 本章的 `run_task`；训练循环 = Part 17

脚本 01 的轨迹级 GRPO。列出你要解决的三个新问题（提示：动作空间是 JSON 调用、观测是工具结果字符串、判分是终态——各对应一个 Part 17 讲过的机制）。

参考资源

- 脚本：`./scripts/02_mini_mcp.py` · `./scripts/03_tau_mini.py`
- MCP 官方规范：[`modelcontextprotocol.io`](<https://modelcontextprotocol.io>)
- A2A 官方站：[`a2a-protocol.org`](<https://a2a-protocol.org>)
- Anthropic 多 agent 研究系统工程博客：[How we built our multi-agent research system](<https://www.anthropic.com/engineering/built-multi-agent-research-system>)（token 消耗约 $15\times$ 的出处）
- Cognition：[Don't Build Multi-Agents](<https://cognition.ai/blog/dont-build-multi-agents>)
- LangChain 博客（Context Engineering / 多 agent 架构系列）：[`blog.langchain.com`](<https://blog.langchain.com>)
- smolagents：[HuggingFace 博客与文档](<https://huggingface.co/docs/smolagents/index>)

- Mario Zechner 极简派：[What I learned building an opinionated and minimal coding agent](https://mariozechner.at/posts/2025-11-30-pi-coding-agent/) ("On AI Agents" 设计笔记) · [badlogic/pi-mono](https://github.com/badlogic/pi-mono)
- SWE-bench 官方：[swebench.com](https://www.swebench.com) (Verified 子集与官方榜单)
- τ -bench / τ^2 -bench： τ^2 -bench 论文 arXiv 2506.07982；GiGPO arXiv 2505.10978；AgentRL arXiv 2510.04206
- Part 17 (训练侧姊妹篇)：../Part17_agentic_rl/tutorial/README.md

学完本章你能...

- 画出 MCP/A2A/AGENTS.md 三层分工图并各举一个真实用例
- 手写 mini-MCP (JSON-RPC 握手/列工具/调工具) 并说出与真实规范的差异
- 按任务特征选框架流派 (状态机/SDK/代码即动作/极简)
- 复述多智能体三方辩论并给出"上下文隔离才是收益来源"的结论
- 正确读 agent 榜单 (锁版本/看脚手架/官方源) 并用 τ -mini 实测 pass^1

课后作业

[Assignment 19](../assignments/assignment_19/)

下一步

Part 19 到此收束：你会用 agent (01 章 loop)、接生态 (02 章协议与框架)、评系统 (τ -mini)。应用线 (A2) 在此闭环；想继续深挖训练侧，回 [Part 17](../Part17_agentic_rl/tutorial/README.md) 用本章的评测当奖励训它。面试准备：[面试指南 § 7b 方向深挖](../docs/llm_interview_guide.md)。

[← 上一章：01 手写 Agent Loop](01_agent_loop.md) | [Part 19 README](README.md)